

LAPORAN PRAKTIKUM

STRUKTUR DATA

“Binary Tree”

disusun Oleh:

Sofian Arba'i (2511533029)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT atas rahmat dan karunia-Nya sehingga laporan praktikum Struktur Data dengan topik *Binary Tree dan Graph Traversal* ini dapat diselesaikan dengan baik. Shalawat serta salam semoga senantiasa tercurahkan kepada Nabi Muhammad SAW.

Laporan ini disusun sebagai bentuk pertanggungjawaban atas pelaksanaan praktikum Struktur Data Pekan 9 yang membahas implementasi Binary Tree beserta operasi traversalnya, serta algoritma penelusuran graf Depth First Search (DFS) dan Breadth First Search (BFS) menggunakan bahasa pemrograman Java.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 8 Juni 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	ii
1.1 Latar Belakang	1
1.2 Tujuan.....	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Pengertian <i>Binary Tree</i>	3
2.1.1 Ciri-Ciri Binary Tree	3
2.1.2 Tree Traversal (Penjelajahan Pohon)	4
2.1.3 Kelebihan dan Kekurangan	4
2.2 Implementasi <i>Binary Tree</i>	5
2.2.1 Class Node_2511533029.....	5
2.2.2 Class Btree_2511533029.....	8
2.2.3 Class BtreeDriver_2511533029	10
2.2.4 Class GraphTraversal_2511533029	13
BAB III PENUTUP	17
3.1 Kesimpulan	17
3.2 Saran.....	17
DAFTAR PUSTAKA	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, pengelolaan data secara efisien merupakan hal yang sangat penting. Salah satu pendekatan yang umum digunakan adalah dengan memanfaatkan struktur data non-linear, seperti pohon (tree) dan graf (graph). Struktur data tersebut mampu merepresentasikan hubungan hierarkis dan relasi antar elemen dengan lebih alami dibandingkan struktur data linear seperti array maupun linked list.

Binary Tree merupakan salah satu bentuk struktur pohon yang paling mendasar, di mana setiap node dapat memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan. Keunggulan Binary Tree terletak pada kemampuannya dalam melakukan operasi pencarian, penyisipan, dan penghapusan data secara efisien. Selain itu, untuk mengakses seluruh elemen dalam pohon, diperlukan teknik penelusuran (traversal) seperti Inorder, Preorder, dan Postorder yang masing-masing menghasilkan urutan kunjungan node yang berbeda sesuai kebutuhan.

Di sisi lain, struktur data graf merepresentasikan hubungan antar objek yang lebih kompleks dan tidak selalu bersifat hierarkis. Graf digunakan secara luas dalam pemodelan jaringan seperti jaringan komputer, peta, dan sistem rekomendasi. Untuk menelusuri seluruh simpul pada graf, digunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS) yang memiliki pendekatan berbeda — DFS menelusuri graf sedalam mungkin secara rekursif, sedangkan BFS menelusuri graf lapis per lapis secara iteratif.

1.2 Tujuan

Tujuan yang diharapkan pada praktikum pekan 9 tentang Binary Tree yaitu sebagai berikut.

1. Memahami konsep dan karakteristik struktur data Binary Tree, termasuk jenis-jenis traversal (Inorder, Preorder, dan Postorder).
2. Mengimplementasikan Binary Tree dalam bahasa pemrograman Java menggunakan pendekatan pemrograman berorientasi objek.
3. Memahami konsep struktur data graf tak berarah dan cara merepresentasikannya menggunakan Adjacency List.
4. Mengimplementasikan algoritma penelusuran graf Depth First Search (DFS) dan Breadth First Search (BFS) dalam bahasa pemrograman Java.

1.3 Manfaat

Manfaat yang didapatkan setelah melakukan praktikum pekan 9 tentang Binary Tree yaitu sebagai berikut.

1. Mahasiswa mampu memahami dan menerapkan struktur data Binary Tree serta berbagai metode traversalnya dalam menyelesaikan permasalahan pemrograman.
2. Mahasiswa dapat mengimplementasikan struktur data graf dan memahami perbedaan pendekatan antara algoritma DFS dan BFS dalam penelusuran graf.
3. Mahasiswa memperoleh kemampuan praktis dalam menulis program Java berbasis struktur data non-linear yang dapat diterapkan pada kasus nyata seperti pemodelan jaringan dan pencarian jalur.
4. Mahasiswa semakin terampil dalam menerapkan konsep pemrograman berorientasi objek, seperti enkapsulasi dan rekursivitas, dalam konteks implementasi struktur data.

BAB II

PEMBAHASAN

2.1 Pengertian *Binary Tree*

Secara sederhana, binary tree (pohon biner) adalah struktur hierarkis yang terdiri dari node-node yang saling terhubung. Setiap node dalam pohon biner memiliki dua anak, yaitu anak kiri dan anak kanan. Masing-masing anak tersebut juga bisa memiliki dua anak lagi, dan begitu seterusnya. Ini membentuk suatu struktur yang bercabang, mirip dengan cabang-cabang pohon.

Keuntungan utama dari struktur ini adalah kemampuannya untuk mengorganisir data dan melakukan berbagai operasi dengan cara yang efisien. Sebagai contoh, pencarian data dalam pohon biner dapat dilakukan lebih cepat karena sifat hierarkinya, yang membantu mempersempit ruang pencarian.

Jadi, dibandingkan dengan struktur data lain yang lebih sederhana, seperti array atau linked list, pohon biner menawarkan keunggulan dalam hal efisiensi. Inti dari binary tree adalah sama, yakni mengorganisasi data dalam bentuk struktur hierarkis yang memudahkan berbagai operasi.

2.1.1 Ciri-Ciri Binary Tree

Berikut beberapa ciri-ciri dari *binary tree* sebagai pemahaman.

1. Node: Setiap elemen dalam *binary tree* disebut node. Node ini menyimpan data dan memiliki dua bagian utama, yaitu data dan *pointer*.
2. Root: Root atau akar adalah node pertama dalam pohon yang menjadi titik awal untuk mencapai semua node lainnya.

3. Anak Kiri dan Anak Kanan: Setiap node dalam pohon biner bisa memiliki dua anak, yaitu anak kiri dan anak kanan. Namun, ada kemungkinan suatu node hanya memiliki satu anak atau bahkan tidak memiliki anak sama sekali (leaf node).
4. Tingkat Kedalaman dan Tinggi Pohon: *Depth* (kedalaman) dari sebuah node adalah panjang jalur dari *root* ke node tersebut. Sedangkan *height* (tinggi) pohon adalah panjang jalur terpanjang dari *root* ke salah satu *leaf node*.

2.1.2 Tree Traversal (Penjelajahan Pohon)

Tree Traversal adalah proses mengunjungi setiap node dalam *Binary Tree* secara sistematis. Terdapat beberapa cara untuk melakukan Tree Traversal, masing-masing menghasilkan urutan kunjungan node yang berbeda.

Tiga jenis Tree Traversal yang paling umum adalah: inorder (kunjungi anak kiri, lalu node itu sendiri, lalu anak kanan), Preorder (kunjungi node itu sendiri, lalu anak kiri, lalu anak kanan), dan Postorder (kunjungi anak kiri, lalu anak kanan, lalu node itu sendiri).

2.1.3 Kelebihan dan Kekurangan

Binary Tree menawarkan beberapa keunggulan yang signifikan dibandingkan struktur data lainnya. Keunggulan tersebut meliputi: pencarian, penyisipan, dan penghapusan data yang efisien (terutama dalam BST), representasi data hierarkis yang alami, dan fleksibilitas dalam implementasi berbagai algoritma.

Namun, Binary Tree juga memiliki beberapa kekurangan. Kekurangan tersebut meliputi: efisiensi yang buruk dalam kasus skewed tree (pohon yang tidak seimbang), kompleksitas implementasi

yang relatif tinggi, dan kebutuhan memori yang lebih besar dibandingkan struktur data linier seperti array atau linked list.

2.2 Implementasi *Binary Tree*

Implementasi *Binary Tree* pada pekan 9 akan dilakukan menggunakan bahasa pemrograman java. Berikut beberapa implementasi dari *Binary Tree*.

2.2.1 Class Node_2511533029

Kode program Node merupakan implementasi dari dasar dari simpul pada struktur data Binary Tree yang menyimpan data serta referensi ke anak kiri dan kanan. Kelas ini menyediakan fungsi untuk mengelola hubungan antar node, melakukan traversal pohon secara rekursif, dan menampilkan struktur pohon dalam bentuk visual. Berikut kode program Node.

```

1. package pekan9_2511533029;
2.
3. public class Node_2511533029 {
4.     int data_3029;        // boleh string
5.     Node_2511533029 left_3029;
6.     Node_2511533029 right_3029;
7.
8.     public Node_2511533029(int data_3029) {
9.         this.data_3029 = data_3029;
10.        left_3029 = null;
11.        right_3029 = null;
12.    }
13.
14.    public void setLeft_3029(Node_2511533029 node_3029)
15.    {
16.        if (left_3029 == null)
17.            left_3029 = node_3029;
18.    }
19.
20.    public void setRight_3029(Node_2511533029 node_3029)
21.    {
22.        if (right_3029 == null)
23.            right_3029 = node_3029;
24.    }
25.
26.    public Node_2511533029 getLeft_3029() {
27.        return left_3029;
28.    }
29.
30.    public Node_2511533029 getRight_3029() {
31.        return right_3029;
32.    }
33.
34.    public void display() {
35.        displayRec();
36.    }
37.
38.    private void displayRec() {
39.        if (left_3029 != null)
40.            displayRec();
41.        System.out.print(data_3029 + " ");
42.        if (right_3029 != null)
43.            displayRec();
44.        System.out.println();
45.    }
46.
47.    public void destroy() {
48.        left_3029 = null;
49.        right_3029 = null;
50.    }
51.
52.    public void reset() {
53.        data_3029 = 0;
54.        left_3029 = null;
55.        right_3029 = null;
56.    }
57.
58.    public void print() {
59.        System.out.println("Node: " + data_3029);
60.    }
61.
62.    public void printRec() {
63.        printRecRec();
64.    }
65.
66.    private void printRecRec() {
67.        if (left_3029 != null)
68.            printRecRec();
69.        print();
70.        if (right_3029 != null)
71.            printRecRec();
72.    }
73.
74.    public void printLevel() {
75.        printLevelRec(0);
76.    }
77.
78.    private void printLevelRec(int level) {
79.        if (level == 0)
80.            print();
81.        else
82.            printLevelRec(level - 1);
83.    }
84.
85.    public void printLevelRec(int level) {
86.        if (level == 0)
87.            print();
88.        else
89.            printLevelRec(level - 1);
90.    }
91.
92.    public void printLevelRec(int level) {
93.        if (level == 0)
94.            print();
95.        else
96.            printLevelRec(level - 1);
97.    }
98.
99.    public void printLevelRec(int level) {
100.        if (level == 0)
101.            print();
102.        else
103.            printLevelRec(level - 1);
104.    }
105.
106.    public void printLevelRec(int level) {
107.        if (level == 0)
108.            print();
109.        else
110.            printLevelRec(level - 1);
111.    }
112.
113.    public void printLevelRec(int level) {
114.        if (level == 0)
115.            print();
116.        else
117.            printLevelRec(level - 1);
118.    }
119.
120.    public void printLevelRec(int level) {
121.        if (level == 0)
122.            print();
123.        else
124.            printLevelRec(level - 1);
125.    }
126.
127.    public void printLevelRec(int level) {
128.        if (level == 0)
129.            print();
130.        else
131.            printLevelRec(level - 1);
132.    }
133.
134.    public void printLevelRec(int level) {
135.        if (level == 0)
136.            print();
137.        else
138.            printLevelRec(level - 1);
139.    }
140.
141.    public void printLevelRec(int level) {
142.        if (level == 0)
143.            print();
144.        else
145.            printLevelRec(level - 1);
146.    }
147.
148.    public void printLevelRec(int level) {
149.        if (level == 0)
150.            print();
151.        else
152.            printLevelRec(level - 1);
153.    }
154.
155.    public void printLevelRec(int level) {
156.        if (level == 0)
157.            print();
158.        else
159.            printLevelRec(level - 1);
160.    }
161.
162.    public void printLevelRec(int level) {
163.        if (level == 0)
164.            print();
165.        else
166.            printLevelRec(level - 1);
167.    }
168.
169.    public void printLevelRec(int level) {
170.        if (level == 0)
171.            print();
172.        else
173.            printLevelRec(level - 1);
174.    }
175.
176.    public void printLevelRec(int level) {
177.        if (level == 0)
178.            print();
179.        else
180.            printLevelRec(level - 1);
181.    }
182.
183.    public void printLevelRec(int level) {
184.        if (level == 0)
185.            print();
186.        else
187.            printLevelRec(level - 1);
188.    }
189.
190.    public void printLevelRec(int level) {
191.        if (level == 0)
192.            print();
193.        else
194.            printLevelRec(level - 1);
195.    }
196.
197.    public void printLevelRec(int level) {
198.        if (level == 0)
199.            print();
200.        else
201.            printLevelRec(level - 1);
202.    }
203.
204.    public void printLevelRec(int level) {
205.        if (level == 0)
206.            print();
207.        else
208.            printLevelRec(level - 1);
209.    }
210.
211.    public void printLevelRec(int level) {
212.        if (level == 0)
213.            print();
214.        else
215.            printLevelRec(level - 1);
216.    }
217.
218.    public void printLevelRec(int level) {
219.        if (level == 0)
220.            print();
221.        else
222.            printLevelRec(level - 1);
223.    }
224.
225.    public void printLevelRec(int level) {
226.        if (level == 0)
227.            print();
228.        else
229.            printLevelRec(level - 1);
230.    }
231.
232.    public void printLevelRec(int level) {
233.        if (level == 0)
234.            print();
235.        else
236.            printLevelRec(level - 1);
237.    }
238.
239.    public void printLevelRec(int level) {
240.        if (level == 0)
241.            print();
242.        else
243.            printLevelRec(level - 1);
244.    }
245.
246.    public void printLevelRec(int level) {
247.        if (level == 0)
248.            print();
249.        else
250.            printLevelRec(level - 1);
251.    }
252.
253.    public void printLevelRec(int level) {
254.        if (level == 0)
255.            print();
256.        else
257.            printLevelRec(level - 1);
258.    }
259.
260.    public void printLevelRec(int level) {
261.        if (level == 0)
262.            print();
263.        else
264.            printLevelRec(level - 1);
265.    }
266.
267.    public void printLevelRec(int level) {
268.        if (level == 0)
269.            print();
270.        else
271.            printLevelRec(level - 1);
272.    }
273.
274.    public void printLevelRec(int level) {
275.        if (level == 0)
276.            print();
277.        else
278.            printLevelRec(level - 1);
279.    }
280.
281.    public void printLevelRec(int level) {
282.        if (level == 0)
283.            print();
284.        else
285.            printLevelRec(level - 1);
286.    }
287.
288.    public void printLevelRec(int level) {
289.        if (level == 0)
290.            print();
291.        else
292.            printLevelRec(level - 1);
293.    }
294.
295.    public void printLevelRec(int level) {
296.        if (level == 0)
297.            print();
298.        else
299.            printLevelRec(level - 1);
300.    }
301.
302.    public void printLevelRec(int level) {
303.        if (level == 0)
304.            print();
305.        else
306.            printLevelRec(level - 1);
307.    }
308.
309.    public void printLevelRec(int level) {
310.        if (level == 0)
311.            print();
312.        else
313.            printLevelRec(level - 1);
314.    }
315.
316.    public void printLevelRec(int level) {
317.        if (level == 0)
318.            print();
319.        else
320.            printLevelRec(level - 1);
321.    }
322.
323.    public void printLevelRec(int level) {
324.        if (level == 0)
325.            print();
326.        else
327.            printLevelRec(level - 1);
328.    }
329.
330.    public void printLevelRec(int level) {
331.        if (level == 0)
332.            print();
333.        else
334.            printLevelRec(level - 1);
335.    }
336.
337.    public void printLevelRec(int level) {
338.        if (level == 0)
339.            print();
340.        else
341.            printLevelRec(level - 1);
342.    }
343.
344.    public void printLevelRec(int level) {
345.        if (level == 0)
346.            print();
347.        else
348.            printLevelRec(level - 1);
349.    }
350.
351.    public void printLevelRec(int level) {
352.        if (level == 0)
353.            print();
354.        else
355.            printLevelRec(level - 1);
356.    }
357.
358.    public void printLevelRec(int level) {
359.        if (level == 0)
360.            print();
361.        else
362.            printLevelRec(level - 1);
363.    }
364.
365.    public void printLevelRec(int level) {
366.        if (level == 0)
367.            print();
368.        else
369.            printLevelRec(level - 1);
370.    }
371.
372.    public void printLevelRec(int level) {
373.        if (level == 0)
374.            print();
375.        else
376.            printLevelRec(level - 1);
377.    }
378.
379.    public void printLevelRec(int level) {
380.        if (level == 0)
381.            print();
382.        else
383.            printLevelRec(level - 1);
384.    }
385.
386.    public void printLevelRec(int level) {
387.        if (level == 0)
388.            print();
389.        else
390.            printLevelRec(level - 1);
391.    }
392.
393.    public void printLevelRec(int level) {
394.        if (level == 0)
395.            print();
396.        else
397.            printLevelRec(level - 1);
398.    }
399.
400.    public void printLevelRec(int level) {
401.        if (level == 0)
402.            print();
403.        else
404.            printLevelRec(level - 1);
405.    }
406.
407.    public void printLevelRec(int level) {
408.        if (level == 0)
409.            print();
410.        else
411.            printLevelRec(level - 1);
412.    }
413.
414.    public void printLevelRec(int level) {
415.        if (level == 0)
416.            print();
417.        else
418.            printLevelRec(level - 1);
419.    }
420.
421.    public void printLevelRec(int level) {
422.        if (level == 0)
423.            print();
424.        else
425.            printLevelRec(level - 1);
426.    }
427.
428.    public void printLevelRec(int level) {
429.        if (level == 0)
430.            print();
431.        else
432.            printLevelRec(level - 1);
433.    }
434.
435.    public void printLevelRec(int level) {
436.        if (level == 0)
437.            print();
438.        else
439.            printLevelRec(level - 1);
440.    }
441.
442.    public void printLevelRec(int level) {
443.        if (level == 0)
444.            print();
445.        else
446.            printLevelRec(level - 1);
447.    }
448.
449.    public void printLevelRec(int level) {
450.        if (level == 0)
451.            print();
452.        else
453.            printLevelRec(level - 1);
454.    }
455.
456.    public void printLevelRec(int level) {
457.        if (level == 0)
458.            print();
459.        else
460.            printLevelRec(level - 1);
461.    }
462.
463.    public void printLevelRec(int level) {
464.        if (level == 0)
465.            print();
466.        else
467.            printLevelRec(level - 1);
468.    }
469.
470.    public void printLevelRec(int level) {
471.        if (level == 0)
472.            print();
473.        else
474.            printLevelRec(level - 1);
475.    }
476.
477.    public void printLevelRec(int level) {
478.        if (level == 0)
479.            print();
480.        else
481.            printLevelRec(level - 1);
482.    }
483.
484.    public void printLevelRec(int level) {
485.        if (level == 0)
486.            print();
487.        else
488.            printLevelRec(level - 1);
489.    }
490.
491.    public void printLevelRec(int level) {
492.        if (level == 0)
493.            print();
494.        else
495.            printLevelRec(level - 1);
496.    }
497.
498.    public void printLevelRec(int level) {
499.        if (level == 0)
500.            print();
501.        else
502.            printLevelRec(level - 1);
503.    }
504.
505.    public void printLevelRec(int level) {
506.        if (level == 0)
507.            print();
508.        else
509.            printLevelRec(level - 1);
510.    }
511.
512.    public void printLevelRec(int level) {
513.        if (level == 0)
514.            print();
515.        else
516.            printLevelRec(level - 1);
517.    }
518.
519.    public void printLevelRec(int level) {
520.        if (level == 0)
521.            print();
522.        else
523.            printLevelRec(level - 1);
524.    }
525.
526.    public void printLevelRec(int level) {
527.        if (level == 0)
528.            print();
529.        else
530.            printLevelRec(level - 1);
531.    }
532.
533.    public void printLevelRec(int level) {
534.        if (level == 0)
535.            print();
536.        else
537.            printLevelRec(level - 1);
538.    }
539.
540.    public void printLevelRec(int level) {
541.        if (level == 0)
542.            print();
543.        else
544.            printLevelRec(level - 1);
545.    }
546.
547.    public void printLevelRec(int level) {
548.        if (level == 0)
549.            print();
550.        else
551.            printLevelRec(level - 1);
552.    }
553.
554.    public void printLevelRec(int level) {
555.        if (level == 0)
556.            print();
557.        else
558.            printLevelRec(level - 1);
559.    }
560.
561.    public void printLevelRec(int level) {
562.        if (level == 0)
563.            print();
564.        else
565.            printLevelRec(level - 1);
566.    }
567.
568.    public void printLevelRec(int level) {
569.        if (level == 0)
570.            print();
571.        else
572.            printLevelRec(level - 1);
573.    }
574.
575.    public void printLevelRec(int level) {
576.        if (level == 0)
577.            print();
578.        else
579.            printLevelRec(level - 1);
580.    }
581.
582.    public void printLevelRec(int level) {
583.        if (level == 0)
584.            print();
585.        else
586.            printLevelRec(level - 1);
587.    }
588.
589.    public void printLevelRec(int level) {
590.        if (level == 0)
591.            print();
592.        else
593.            printLevelRec(level - 1);
594.    }
595.
596.    public void printLevelRec(int level) {
597.        if (level == 0)
598.            print();
599.        else
600.            printLevelRec(level - 1);
601.    }
602.
603.    public void printLevelRec(int level) {
604.        if (level == 0)
605.            print();
606.        else
607.            printLevelRec(level - 1);
608.    }
609.
610.    public void printLevelRec(int level) {
611.        if (level == 0)
612.            print();
613.        else
614.            printLevelRec(level - 1);
615.    }
616.
617.    public void printLevelRec(int level) {
618.        if (level == 0)
619.            print();
620.        else
621.            printLevelRec(level - 1);
622.    }
623.
624.    public void printLevelRec(int level) {
625.        if (level == 0)
626.            print();
627.        else
628.            printLevelRec(level - 1);
629.    }
630.
631.    public void printLevelRec(int level) {
632.        if (level == 0)
633.            print();
634.        else
635.            printLevelRec(level - 1);
636.    }
637.
638.    public void printLevelRec(int level) {
639.        if (level == 0)
640.            print();
641.        else
642.            printLevelRec(level - 1);
643.    }
644.
645.    public void printLevelRec(int level) {
646.        if (level == 0)
647.            print();
648.        else
649.            printLevelRec(level - 1);
650.    }
651.
652.    public void printLevelRec(int level) {
653.        if (level == 0)
654.            print();
655.        else
656.            printLevelRec(level - 1);
657.    }
658.
659.    public void printLevelRec(int level) {
660.        if (level == 0)
661.            print();
662.        else
663.            printLevelRec(level - 1);
664.    }
665.
666.    public void printLevelRec(int level) {
667.        if (level == 0)
668.            print();
669.        else
670.            printLevelRec(level - 1);
671.    }
672.
673.    public void printLevelRec(int level) {
674.        if (level == 0)
675.            print();
676.        else
677.            printLevelRec(level - 1);
678.    }
679.
680.    public void printLevelRec(int level) {
681.        if (level == 0)
682.            print();
683.        else
684.            printLevelRec(level - 1);
685.    }
686.
687.    public void printLevelRec(int level) {
688.        if (level == 0)
689.            print();
690.        else
691.            printLevelRec(level - 1);
692.    }
693.
694.    public void printLevelRec(int level) {
695.        if (level == 0)
696.            print();
697.        else
698.            printLevelRec(level - 1);
699.    }
700.
701.    public void printLevelRec(int level) {
702.        if (level == 0)
703.            print();
704.        else
705.            printLevelRec(level - 1);
706.    }
707.
708.    public void printLevelRec(int level) {
709.        if (level == 0)
710.            print();
711.        else
712.            printLevelRec(level - 1);
713.    }
714.
715.    public void printLevelRec(int level) {
716.        if (level == 0)
717.            print();
718.        else
719.            printLevelRec(level - 1);
720.    }
721.
722.    public void printLevelRec(int level) {
723.        if (level == 0)
724.            print();
725.        else
726.            printLevelRec(level - 1);
727.    }
728.
729.    public void printLevelRec(int level) {
730.        if (level == 0)
731.            print();
732.        else
733.            printLevelRec(level - 1);
734.    }
735.
736.    public void printLevelRec(int level) {
737.        if (level == 0)
738.            print();
739.        else
740.            printLevelRec(level - 1);
741.    }
742.
743.    public void printLevelRec(int level) {
744.        if (level == 0)
745.            print();
746.        else
747.            printLevelRec(level - 1);
748.    }
749.
750.    public void printLevelRec(int level) {
751.        if (level == 0)
752.            print();
753.        else
754.            printLevelRec(level - 1);
755.    }
756.
757.    public void printLevelRec(int level) {
758.        if (level == 0)
759.            print();
760.        else
761.            printLevelRec(level - 1);
762.    }
763.
764.    public void printLevelRec(int level) {
765.        if (level == 0)
766.            print();
767.        else
768.            printLevelRec(level - 1);
769.    }
770.
771.    public void printLevelRec(int level) {
772.        if (level == 0)
773.            print();
774.        else
775.            printLevelRec(level - 1);
776.    }
777.
778.    public void printLevelRec(int level) {
779.        if (level == 0)
780.            print();
781.        else
782.            printLevelRec(level - 1);
783.    }
784.
785.    public void printLevelRec(int level) {
786.        if (level == 0)
787.            print();
788.        else
789.            printLevelRec(level - 1);
790.    }
791.
792.    public void printLevelRec(int level) {
793.        if (level == 0)
794.            print();
795.        else
796.            printLevelRec(level - 1);
797.    }
798.
799.    public void printLevelRec(int level) {
800.        if (level == 0)
801.            print();
802.        else
803.            printLevelRec(level - 1);
804.    }
805.
806.    public void printLevelRec(int level) {
807.        if (level == 0)
808.            print();
809.        else
810.            printLevelRec(level - 1);
811.    }
812.
813.    public void printLevelRec(int level) {
814.        if (level == 0)
815.            print();
816.        else
817.            printLevelRec(level - 1);
818.    }
819.
820.    public void printLevelRec(int level) {
821.        if (level == 0)
822.            print();
823.        else
824.            printLevelRec(level - 1);
825.    }
826.
827.    public void printLevelRec(int level) {
828.        if (level == 0)
829.            print();
830.        else
831.            printLevelRec(level - 1);
832.    }
833.
834.    public void printLevelRec(int level) {
835.        if (level == 0)
836.            print();
837.        else
838.            printLevelRec(level - 1);
839.    }
840.
841.    public void printLevelRec(int level) {
842.        if (level == 0)
843.            print();
844.        else
845.            printLevelRec(level - 1);
846.    }
847.
848.    public void printLevelRec(int level) {
849.        if (level == 0)
850.            print();
851.        else
852.            printLevelRec(level - 1);
853.    }
854.
855.    public void printLevelRec(int level) {
856.        if (level == 0)
857.            print();
858.        else
859.            printLevelRec(level - 1);
860.    }
861.
862.    public void printLevelRec(int level) {
863.        if (level == 0)
864.            print();
865.        else
866.            printLevelRec(level - 1);
867.    }
868.
869.    public void printLevelRec(int level) {
870.        if (level == 0)
871.            print();
872.        else
873.            printLevelRec(level - 1);
874.    }
875.
876.    public void printLevelRec(int level) {
877.        if (level == 0)
878.            print();
879.        else
880.            printLevelRec(level - 1);
881.    }
882.
883.    public void printLevelRec(int level) {
884.        if (level == 0)
885.            print();
886.        else
887.            printLevelRec(level - 1);
888.    }
889.
890.    public void printLevelRec(int level) {
891.        if (level == 0)
892.            print();
893.        else
894.            printLevelRec(level - 1);
895.    }
896.
897.    public void printLevelRec(int level) {
898.        if (level == 0)
899.            print();
900.        else
901.            printLevelRec(level - 1);
902.    }
903.
904.    public void printLevelRec(int level) {
905.        if (level == 0)
906.            print();
907.        else
908.            printLevelRec(level - 1);
909.    }
910.
911.    public void printLevelRec(int level) {
912.        if (level == 0)
913.            print();
914.        else
915.            printLevelRec(level - 1);
916.    }
917.
918.    public void printLevelRec(int level) {
919.        if (level == 0)
920.            print();
921.        else
922.            printLevelRec(level - 1);
923.    }
924.
925.    public void printLevelRec(int level) {
926.        if (level == 0)
927.            print();
928.        else
929.            printLevelRec(level - 1);
930.    }
931.
932.    public void printLevelRec(int level) {
933.        if (level == 0)
934.            print();
935.        else
936.            printLevelRec(level - 1);
937.    }
938.
939.    public void printLevelRec(int level) {
940.        if (level == 0)
941.            print();
942.        else
943.            printLevelRec(level - 1);
944.    }
945.
946.    public void printLevelRec(int level) {
947.        if (level == 0)
948.            print();
949.        else
950.            printLevelRec(level - 1);
951.    }
952.
953.    public void printLevelRec(int level) {
954.        if (level == 0)
955.            print();
956.        else
957.            printLevelRec(level - 1);
958.    }
959.
960.    public void printLevelRec(int level) {
961.        if (level == 0)
962.            print();
963.        else
964.            printLevelRec(level - 1);
965.    }
966.
967.    public void printLevelRec(int level) {
968.        if (level == 0)
969.            print();
970.        else
971.            printLevelRec(level - 1);
972.    }
973.
974.    public void printLevelRec(int level) {
975.        if (level == 0)
976.            print();
977.        else
978.            printLevelRec(level - 1);
979.    }
980.
981.    public void printLevelRec(int level) {
982.        if (level == 0)
983.            print();
984.        else
985.            printLevelRec(level - 1);
986.    }
987.
988.    public void printLevelRec(int level) {
989.        if (level == 0)
990.            print();
991.        else
992.            printLevelRec(level - 1);
993.    }
994.
995.    public void printLevelRec(int level) {
996.        if (level == 0)
997.            print();
998.        else
999.            printLevelRec(level - 1);
1000.    }

```



```

26.     }
27.
28.     public Node_2511533029 getRight_3029() {
29.         return right_3029;
30.     }
31.
32.     public int getData_3029() {
33.         return data_3029;
34.     }
35.
36.     public void setData_3029(int data_3029) {
37.         this.data_3029 = data_3029;
38.     }
39.
40.     void printPreorder_3029(Node_2511533029 node_3029) {
41.         if (node_3029 == null)
42.             return;
43.
44.         System.out.print(node_3029.data_3029 + " ");
45.         printPreorder_3029(node_3029.left_3029);
46.         System.out.print(node_3029.data_3029 + " ");
47.     }
48.
49.     void printPostorder_3029(Node_2511533029 node_3029)
50. {
51.     if (node_3029 == null)
52.         return;
53.
54.     printPostorder_3029(node_3029.left_3029);
55.     printPostorder_3029(node_3029.right_3029);
56.     System.out.print(node_3029.data_3029 + " ");
57. }
58. void printInorder_3029(Node_2511533029 node_3029) {
59.     if (node_3029 == null)
60.         return;
61.
62.     printInorder_3029(node_3029.left_3029);
63.     System.out.print(node_3029.data_3029 + " ");
64.     printInorder_3029(node_3029.right_3029);
65. }
66.
67.     public String print_3029() {
68.         return this.print_3029("", true, "");
69.     }
70.
71.     public String print_3029(String prefix_3029, boolean
isTail_3029, String sb_3029) {
72.         if (right_3029 != null) {
73.             right_3029.print_3029(
74.                 prefix_3029 + (isTail_3029 ? "|"
" : " "),
75.                 false,
76.                 sb_3029

```

```

77.         );
78.     }
79.
80.     System.out.println(
81.         prefix_3029 + (isTail_3029 ? "\\--" :
82.         "/--") + data_3029
83.     );
84.     if (left_3029 != null) {
85.         left_3029.print_3029(
86.             prefix_3029 + (isTail_3029 ? "
87.             " : "| "),
88.             true,
89.             sb_3029
90.         );
91.     }
92.     return sb_3029;
93. }
94. }

```

Program `Node_2511533029` merupakan kelas yang digunakan untuk merepresentasikan sebuah simpul (node) pada struktur data Binary Tree. Setiap node memiliki tiga atribut utama, yaitu `data_3029` untuk menyimpan nilai data bertipe integer, `left_3029` untuk menyimpan referensi ke anak kiri, dan `right_3029` untuk menyimpan referensi ke anak kanan. Konstruktor kelas digunakan untuk menginisialisasi nilai data node saat objek dibuat, sedangkan atribut anak kiri dan kanan diatur bernilai null karena node belum memiliki hubungan dengan node lain.

Kelas ini juga menyediakan beberapa metode untuk mengelola data dan hubungan antar node, seperti `setLeft_3029()` dan `setRight_3029()` yang digunakan untuk menghubungkan node dengan anak kiri atau kanan, serta metode `getLeft_3029()`, `getRight_3029()`, `getData_3029()`, dan `setData_3029()` yang berfungsi sebagai accessor dan mutator. Dengan adanya metode-metode tersebut, data dan hubungan antar node dapat diakses maupun diubah dengan lebih terstruktur sesuai prinsip enkapsulasi dalam pemrograman berorientasi objek.

Selain itu, kelas ini mengimplementasikan tiga metode traversal Binary Tree, yaitu `printPreorder_3029()`, `printInorder_3029()`, dan `printPostorder_3029()` yang menggunakan teknik rekursif untuk mengunjungi setiap node pada pohon dengan urutan yang berbeda. Terdapat pula metode `print_3029()` yang berfungsi menampilkan struktur pohon secara visual dalam bentuk hierarki sehingga hubungan antara root, anak kiri, dan anak kanan dapat dilihat dengan lebih jelas. Dengan demikian, kelas ini tidak hanya menyimpan data node, tetapi juga menyediakan operasi dasar untuk penelusuran dan visualisasi pohon biner.

2.2.2 Class Btree_2511533029

Class selanjutnya yaitu sebuah class yang berfungsi sebagai pengelola struktur data Binary Tree yang menyediakan berbagai operasi dasar, seperti pencarian data, traversal pohon, pengecekan kondisi pohon, penghitungan jumlah node, dan visualisasi struktur pohon. Berikut kode program Btree (Binary Tree).

```

1.  package pekan9_2511533029;
2.
3.  public class BTree_2511533029 {
4.      private Node_2511533029 root_3029;
5.      private Node_2511533029 currentNode_3029;
6.
7.      public BTree_2511533029() {
8.          root_3029 = null;
9.      }
10.
11.     public boolean search_3029(int data_3029) {
12.         return search_3029(root_3029, data_3029);
13.     }
14.
15.     private boolean search_3029(Node_2511533029
node_3029, int data_3029) {
16.         if (node_3029.getData_3029() == data_3029)
17.             return true;
18.
19.         if (node_3029.getLeft_3029() != null)
20.             return true;
21.
22.         if (node_3029.getRight_3029() != null)

```

```

23.         if
24.         (search_3029(node_3029.getRight_3029(), data_3029))
25.             return true;
26.
27.         return false;
28.     }
29.
30.     public void printInorder_3029() {
31.         root_3029.printInorder_3029(root_3029);
32.     }
33.
34.     public void printPreorder_3029() {
35.         root_3029.printPreorder_3029(root_3029);
36.     }
37.
38.     public void printPostorder_3029() {
39.         root_3029.printPostorder_3029(root_3029);
40.     }
41.
42.     public Node_2511533029 getRoot_3029() {
43.         return root_3029;
44.     }
45.
46.     public boolean isEmpty_3029() {
47.         return root_3029 == null;
48.     }
49.
50.     public int countNodes_3029() {
51.         return countNodes_3029(root_3029);
52.     }
53.
54.     private int countNodes_3029(Node_2511533029
55. node_3029) {
56.         int count_3029 = 1;
57.
58.         if (node_3029 == null) {
59.             return 0;
60.         } else {
61.             count_3029 +=
62. countNodes_3029(node_3029.getLeft_3029());
63.             count_3029 +=
64. countNodes_3029(node_3029.getRight_3029());
65.             return count_3029;
66.         }
67.     }
68.
69.     public void print_3029() {
70.         root_3029.print_3029();
71.     }
72.
73.     public Node_2511533029 getCurrent_3029() {
74.         return currentNode_3029;
75.     }

```

```

73.     public void setCurrent_3029(Node_2511533029
node_3029) {
74.         this.currentNode_3029 = node_3029;
75.     }
76.
77.     public void setRoot_3029(Node_2511533029 root_3029)
{
78.         this.root_3029 = root_3029;
79.     }
80. }

```

Program BTree_2511533029 merupakan kelas yang digunakan untuk merepresentasikan dan mengelola struktur data Binary Tree. Kelas ini memiliki atribut root_3029 sebagai akar pohon dan currentNode_3029 sebagai penunjuk node yang sedang aktif. Konstruktor BTree_2511533029() menginisialisasi nilai root_3029 menjadi null, yang menunjukkan bahwa pohon masih kosong. Selain itu, kelas ini menyediakan metode getRoot_3029(), setRoot_3029(), getCurrent_3029(), dan setCurrent_3029() untuk mengakses dan mengubah nilai root maupun node yang sedang digunakan dalam proses pengelolaan pohon.

Kelas ini juga menyediakan berbagai operasi dasar pada Binary Tree, seperti pencarian data melalui metode search_3029(), pengecekan kondisi pohon kosong dengan isEmpty_3029(), serta perhitungan jumlah node menggunakan metode rekursif countNodes_3029(). Selain itu, terdapat metode printInorder_3029(), printPreorder_3029(), dan printPostorder_3029() yang digunakan untuk melakukan traversal pohon dengan urutan yang berbeda, serta metode print_3029() untuk menampilkan struktur pohon secara visual. Dengan adanya metode-metode tersebut, kelas BTree_2511533029 berfungsi sebagai pengelola utama berbagai operasi yang dapat dilakukan pada struktur data Binary Tree.

2.2.3 Class BtreeDriver_2511533029

Kode program dalam bahasa java berikutnya adalah kode program yang digunakan untuk membangun, menghubungkan, dan menguji sebuah struktur Binary Tree. Program menunjukkan proses pembentukan pohon dari beberapa node, perhitungan jumlah simpul, serta traversal Inorder, Preorder, dan Postorder. Berikut kode program java Binary Tree Driver.

```

1. package pekan9_2511533029;
2.
3. public class BTreeDriver_2511533029 {
4.     public static void main(String[] args) {
5.         // membuat pohon
6.         BTree_2511533029 tree_3029 = new
BTree_2511533029();
7.         System.out.print("Jumlah Simpul awal pohon:
");
8.
9.         System.out.println(tree_3029.countNodes_3029());
10.
11.         // menambahkan simpul data 1
Node_2511533029 root_3029 = new
Node_2511533029(1);
12.
13.         // menjadikan simpul 1 sebagai root
tree_3029.setRoot_3029(root_3029);
14.         System.out.println("Jumlah simpul jika hanya
ada root");
15.
16.         System.out.println(tree_3029.countNodes_3029());
17.
18.         Node_2511533029 node2_3029 = new
Node_2511533029(2);
19.         Node_2511533029 node3_3029 = new
Node_2511533029(3);
20.         Node_2511533029 node4_3029 = new
Node_2511533029(4);
21.         Node_2511533029 node5_3029 = new
Node_2511533029(5);
22.         Node_2511533029 node6_3029 = new
Node_2511533029(6);
23.         Node_2511533029 node7_3029 = new
Node_2511533029(7);
24.         Node_2511533029 node8_3029 = new
Node_2511533029(8);
25.         Node_2511533029 node9_3029 = new
Node_2511533029(9);
26.
27.         root_3029.setLeft_3029(node2_3029);
28.         node2_3029.setLeft_3029(node4_3029);
29.         node2_3029.setRight_3029(node5_3029);

```

```

30.         node4_3029.setRight_3029(node8_3029);
31.
32.         root_3029.setRight_3029(node3_3029);
33.         node3_3029.setLeft_3029(node6_3029);
34.         node3_3029.setRight_3029(node7_3029);
35.         node6_3029.setLeft_3029(node9_3029);
36.
37.         // set root_3029
38.
39.         tree_3029.setCurrent_3029(tree_3029.getRoot_3029());
40.         System.out.println("menampilkan simpul
    terakhir:");
41.
42.         System.out.println(tree_3029.getCurrent_3029().getDa
    ta_3029());
43.
44.         System.out.println("Jumlah simpul; setelah
    simpul 7 ditambahkan");
45.
46.         System.out.println(tree_3029.countNodes_3029());
47.
48.         System.out.println("InOrder: ");
49.         tree_3029.printInorder_3029();
50.
51.         System.out.println("\nPreorder: ");
52.         tree_3029.printPreorder_3029();
53.
54.         System.out.println("\nPostorder: ");
55.         tree_3029.printPostorder_3029();
56.
57.         System.out.println("\nMenampilkan simpul
    dalam bentuk pohon");
58.         tree_3029.print_3029();
59.     }
60. }

```

Program BTreeDriver_2511533029 merupakan kelas utama yang digunakan untuk menguji implementasi struktur data Binary Tree. Pada awal program, dibuat sebuah objek BTree_2511533029 yang merepresentasikan pohon biner. Program kemudian menampilkan jumlah simpul awal yang masih bernilai 0 karena pohon belum memiliki node. Selanjutnya dibuat sebuah node dengan nilai 1 yang dijadikan sebagai root menggunakan metode setRoot_3029(), kemudian jumlah simpul pada pohon ditampilkan kembali untuk menunjukkan bahwa pohon telah memiliki satu simpul.

Setelah root dibuat, program menambahkan beberapa node baru dengan nilai 2 hingga 9 dan menghubungkannya menggunakan metode `setLeft_3029()` dan `setRight_3029()`. Hubungan antar node tersebut membentuk struktur Binary Tree yang memiliki cabang kiri dan kanan dengan beberapa tingkat kedalaman. Setelah seluruh node terhubung, root pohon disimpan sebagai `currentNode_3029` menggunakan metode `setCurrent_3029()`, kemudian data dari node saat ini ditampilkan untuk memastikan bahwa root telah berhasil ditetapkan sebagai node aktif.

Selanjutnya program melakukan beberapa operasi pada Binary Tree, seperti menghitung jumlah seluruh simpul menggunakan metode `countNodes_3029()`, serta menampilkan isi pohon menggunakan tiga metode traversal yaitu Inorder, Preorder, dan Postorder. Selain itu, program juga memanggil metode `print_3029()` untuk menampilkan struktur pohon secara visual dalam bentuk hierarki.

2.2.4 Class GraphTraversal_2511533029

Kode program `GraphTraversal_2511533029` menunjukkan implementasi graf tak berarah menggunakan representasi *Adjacency List* serta penerapan dua algoritma penelusuran graf, yaitu Depth First Search (DFS) dan Breadth First Search (BFS). Program ini memperlihatkan bagaimana simpul dan sisi pada graf dibentuk, ditampilkan, serta ditelusuri menggunakan pendekatan rekursif dan iteratif. Berikut kode program Graph Traversal.

```
1. package pekan9_2511533029;
2.
3. import java.util.*;
4. public class GraphTraversal_2511533029 {
5.     private Map<String, List<String>> graph_3029 = new
    HashMap<>();
6.
7.     // menambahkan edge graf tak terarah
8.     public void addedge_3029 (String node1_3029, String
    node2_3029) {
```



```

9.         graph_3029.putIfAbsent (node1_3029, new
ArrayList<>());
10.        graph_3029.putIfAbsent (node2_3029, new
ArrayList<>());
11.        graph_3029.get (node1_3029).add(node2_3029);
12.        graph_3029.get(node2_3029).add(node1_3029);
13.    }
14.    // menampilkan graf awal
15.    public void printGraph_3029() {
16.        System.out.println("Graf awal (adjacency
List): ");
17.        for (String node_3029 : graph_3029.keySet())
{
18.            System.out.print(node_3029 + " -> ");
19.            List<String> neighbors_3029 =
graph_3029.get(node_3029);
20.            System.out.println(String.join(", ",
neighbors_3029));
21.        }
22.        System.out.println();
23.    }
24.
25.    // DFS rekursif
26.    public void dfs_3029 (String start_3029) {
27.        Set<String> visited_3029 = new HashSet<>();
28.        System.out.println("Penelusuran DFS: ");
29.        dfsHelper_3029 (start_3029, visited_3029);
30.        System.out.println();
31.    }
32.    private void dfsHelper_3029(String current_3029,
Set<String> visited) {
33.        if (visited.contains(current_3029))
return;
34.        visited.add(current_3029);
35.        System.out.print(current_3029 + " ");
36.        for (String neighbor_3029 :
graph_3029.getDefault(current_3029, new ArrayList<>()))
{
37.            dfsHelper_3029(neighbor_3029, visited);
38.        }
39.    }
40.
41.    // BFS Iteratif
42.    public void bfs_3029(String start_3029) {
43.        Set<String> visited_3029 = new HashSet<>();
44.        Queue<String> queue_3029 = new LinkedList<>();
45.
46.        queue_3029.add(start_3029);
47.        visited_3029.add(start_3029);
48.
49.        System.out.println("Penelusuran BFS:");
50.
51.        while (!queue_3029.isEmpty()) {
52.            String current_3029 = queue_3029.poll();
53.            System.out.print(current_3029 + " ");

```

```

54.         for (String neighbor_3029 :
55. graph_3029.getDefault(current_3029, new ArrayList<>()))
56.     {
57.         if
58.     (!visited_3029.contains(neighbor_3029)) {
59.             queue_3029.add(neighbor_3029);
60.             visited_3029.add(neighbor_3029);
61.         }
62.     }
63.     System.out.println();
64. }
65. // Main
66. public static void main(String[] args_3029) {
67.
68.     GraphTraversal_2511533029 graph_3029 = new
69.     GraphTraversal_2511533029();
70.
71.     // Contoh graf: A-B, A-C, B-D, B-E
72.     graph_3029.addedge_3029("A", "B");
73.     graph_3029.addedge_3029("A", "C");
74.     graph_3029.addedge_3029("B", "D");
75.     graph_3029.addedge_3029("B", "E");
76.
77.     // Cetak graf awal
78.     System.out.println("Graf Awal adalah:");
79.     graph_3029.printGraph_3029();
80.
81.     // Lakukan penelusuran
82.     graph_3029.dfs_3029("A");
83.     graph_3029.bfs_3029("A");
84. }

```

Program GraphTraversal_2511533029 merupakan implementasi struktur data graf tak berarah (undirected graph) menggunakan representasi Adjacency List. Graf disimpan dalam variabel graph_3029 yang bertipe Map<String, List<String>>, di mana setiap simpul memiliki daftar simpul tetangga yang terhubung dengannya. Metode addedge_3029() digunakan untuk menambahkan sisi (edge) antara dua simpul dan karena graf bersifat tak berarah, hubungan ditambahkan ke kedua arah. Selain itu, metode printGraph_3029() berfungsi menampilkan seluruh simpul beserta daftar tetangganya sehingga struktur graf dapat dilihat dengan jelas.

Program ini juga mengimplementasikan dua algoritma penelusuran graf, yaitu Depth First Search (DFS) dan Breadth First Search (BFS). Metode `dfs_3029()` menggunakan pendekatan rekursif dengan bantuan `dfsHelper_3029()` untuk menelusuri graf sedalam mungkin sebelum kembali ke simpul sebelumnya, sedangkan metode `bfs_3029()` menggunakan struktur data Queue untuk menelusuri graf secara bertahap berdasarkan tingkat kedekatan simpul. Pada metode `main()`, dibuat sebuah graf sederhana dengan simpul A, B, C, D, dan E, kemudian graf ditampilkan dan dilakukan penelusuran menggunakan DFS serta BFS yang dimulai dari simpul A.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilaksanakan, dapat disimpulkan bahwa Binary Tree merupakan struktur data hierarkis yang efisien di mana setiap node memiliki paling banyak dua anak, yaitu anak kiri dan anak kanan. Dibandingkan struktur data linear seperti array dan linked list, Binary Tree menawarkan keunggulan dalam operasi pencarian, penyisipan, dan penghapusan data. Dalam implementasinya menggunakan Java, Binary Tree dibangun melalui kelas Node yang menyimpan data serta referensi ke anak kiri dan kanan, serta kelas BTree sebagai pengelola operasi-operasi pada pohon tersebut.

Selain itu, terdapat tiga metode traversal yang dapat digunakan untuk mengunjungi seluruh node pada Binary Tree, yaitu Inorder, Preorder, dan Postorder. Masing-masing metode menghasilkan urutan kunjungan node yang berbeda dan dapat dipilih sesuai kebutuhan. Di sisi lain, struktur data graf tak berarah dapat direpresentasikan menggunakan Adjacency List dan ditelusuri menggunakan dua algoritma, yaitu Depth First Search (DFS) dan Breadth First Search (BFS).

3.2 Saran

Dalam mempelajari dan mengimplementasikan struktur data Binary Tree, disarankan agar memperhatikan kondisi keseimbangan pohon sehingga tidak terbentuk *skewed tree* yang dapat menurunkan efisiensi operasi secara signifikan. Selain itu, pemilihan algoritma penelusuran graf sebaiknya disesuaikan dengan kebutuhan, di mana DFS lebih cocok untuk penelusuran jalur yang dalam, sedangkan BFS lebih tepat digunakan untuk menemukan jalur terpendek.

DAFTAR PUSTAKA

- [1] Rifka Amalia, “Binary Tree adalah: Ciri-ciri, Fungsi, Jenis Struktur Data,”. [Daring]. Tersedia pada: [Binary Tree adalah: Ciri-ciri, Fungsi, Jenis Struktur Data](#) [Diakses: 8-Juni-2026].
- [2] Nirwana Ismail, “Mengenal Binary Tree: Struktur Data Hierarkis yang Penting dalam Pemrograman,”. [Daring]. Tersedia pada: [Mengenal Binary Tree: Struktur Data Hierarkis yang Penting dalam Pemrograman](#) [Diakses: 8-Juni-2026].

LAPORAN TUGAS PRAKTIKUM

Struktur Data

Pekan 9 (*Binary Tree*)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Deskripsi Tugas

Mahasiswa diminta untuk mengimplementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI. Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph. Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa:

1. Gedung Perkuliahan
2. Fakultas
3. Universitas
4. Sekolah
5. Rumah Sakit
6. Mall
7. Tempat Wisata
8. Bandara Atau lokasi lainnya

Program harus mampu:

1. Menampilkan graph dalam bentuk visual
2. Menentukan titik awal Start
3. Menentukan titik tujuan Goal
4. Melakukan pencarian menggunakan BFS
5. Melakukan pencarian menggunakan DFS
6. Menampilkan jalur yang ditemukan
7. Menampilkan urutan simpul yang dikunjungi
8. Urutan node yang dikunjungi Jumlah node yang dieksplorasi

Jawaban

1. Kode Program (Penjelasan)

Program PetaSekolah_2511533029 merupakan aplikasi berbasis Java Swing yang digunakan untuk mensimulasikan pencarian jalur antar lokasi di lingkungan sekolah menggunakan algoritma Breadth First Search (BFS) dan Depth First Search (DFS). Program ini memodelkan peta sekolah dalam bentuk graf yang terdiri dari 10 node, yaitu Gerbang, Pos Satpam, Perpustakaan, Lab Komputer, Kelas 10, Kelas 11, Kelas 12, Kantin, Lapangan, dan Kantor Guru, serta 15 edge yang menghubungkan setiap lokasi. Antarmuka program menyediakan pilihan lokasi awal dan lokasi tujuan melalui ComboBox, serta tombol BFS, DFS, dan Reset untuk menjalankan proses pencarian. Algoritma BFS bekerja dengan menelusuri node berdasarkan tingkat kedekatan sehingga dapat menemukan jalur terpendek, sedangkan DFS menelusuri node secara mendalam hingga mencapai tujuan. Hasil pencarian ditampilkan dalam bentuk jalur yang ditemukan, urutan node yang dikunjungi, dan jumlah node yang dieksplorasi selama proses pencarian. Selain itu, program juga menyediakan fitur reset untuk mengembalikan tampilan ke kondisi awal sehingga pengguna dapat melakukan pencarian ulang dengan mudah. Dengan demikian, program ini dapat digunakan sebagai media pembelajaran untuk memahami penerapan struktur data graf serta algoritma BFS dan DFS dalam pencarian jalur.

1.1 Class PetaSekolah_2511533029

Berikut kode program dari class peta sekolah

```
1. package pekan9_2511533029;
2.
3. import javax.swing.*;
4. import javax.swing.border.*;
5. import java.awt.*;
6. import java.util.Arrays;
7. import java.util.ArrayList;
8. import java.util.LinkedList;
9. import java.util.List;
10. import java.util.Queue;
```



```

11. import java.util.Stack;
12.
13. public class PetaSekolah_2511533029 extends JFrame {
14.
15.     // ===== 10 NODE, 15 EDGE =====
16.     private final String[] nodes_3029 = {
17.         "Gerbang",           // 0
18.         "PosSatpam",         // 1
19.         "Perpustakaan",     // 2
20.         "LabKomputer",       // 3
21.         "Kelas10",          // 4
22.         "Kelas11",          // 5
23.         "Kelas12",          // 6
24.         "Kantin",            // 7
25.         "Lapangan",          // 8
26.         "KantorGuru"         // 9
27.     };
28.     // 15 edge
29.     private final int[][] edges_3029 = {
30.
31.         {0,1},
32.         {0,8},
33.
34.         {1,2},
35.         {1,9},
36.
37.         {2,3},
38.         {2,4},
39.
40.         {3,5},
41.         {3,6},
42.
43.         {4,5},
44.         {5,6},
45.
46.         {8,7},
47.         {7,6},
48.
49.         {9,4},
50.         {9,2},
51.         {7,4}
52.     };
53.     private JComboBox<String> cbStart_3029, cbGoal_3029;
54.     private JTextArea taGraph_3029, taResult_3029;
55.
56.     public PetaSekolah_2511533029() {
57.         setTitle("Peta Kampus - BFS & DFS | 2511533029");
58.         setDefaultCloseOperation(EXIT_ON_CLOSE);
59.         setLayout(new BorderLayout(4, 4));
60.
61.         buildTop_3029();
62.         buildGraph_3029();
63.         buildResult_3029();
64.

```

```

65.         setSize(750, 520);
66.         setLocationRelativeTo(null);
67.         setVisible(true);
68.     }
69.
70.     private void buildTop_3029() {
71.         JPanel north_3029 = new JPanel(new BorderLayout());
72.
73.         JLabel title_3029 = new JLabel("PENCARIAN JALUR
MENGUNAKAN BFS DAN DFS", SwingConstants.CENTER);
74.         title_3029.setFont(new Font("Dialog", Font.BOLD, 15));
75.         title_3029.setForeground(Color.WHITE);
76.         title_3029.setOpaque(true);
77.         title_3029.setBackground(new Color(50, 80, 130));
78.         title_3029.setBorder(BorderFactory.createEmptyBorder(8,
8, 8, 8));
79.
80.         JPanel ctrl_3029 = new JPanel(new
FlowLayout(FlowLayout.LEFT, 10, 6));
81.         ctrl_3029.setBackground(new Color(230, 230, 230));
82.
83.         cbStart_3029 = new JComboBox<>(nodes_3029);
84.         cbStart_3029.setSelectedIndex(0);
85.         cbGoal_3029 = new JComboBox<>(nodes_3029);
86.         cbGoal_3029.setSelectedIndex(6);
87.
88.         JButton btnBFS_3029 = buatTombol_3029("[ BFS ]", new
Color(60, 160, 60));
89.         JButton btnDFS_3029 = buatTombol_3029("[ DFS ]", new
Color(200, 130, 0));
90.         JButton btnReset_3029 = buatTombol_3029("[ RESET ]", new
Color(190, 40, 40));
91.
92.         btnBFS_3029.addActionListener(e -> {
93.             resetGraph_2511533029();
94.             BFS_2511533029(cbStart_3029.getSelectedIndex(),
cbGoal_3029.getSelectedIndex());
95.         });
96.         btnDFS_3029.addActionListener(e -> {
97.             resetGraph_2511533029();
98.             DFS_2511533029(cbStart_3029.getSelectedIndex(),
cbGoal_3029.getSelectedIndex());
99.         });
100.        btnReset_3029.addActionListener(e ->
resetGraph_2511533029());
101.
102.        ctrl_3029.add(new JLabel("Lokasi Awal :"));
103.        ctrl_3029.add(cbStart_3029);
104.        ctrl_3029.add(new JLabel("Lokasi Tujuan :"));
105.        ctrl_3029.add(cbGoal_3029);
106.        ctrl_3029.add(btnBFS_3029);
107.        ctrl_3029.add(btnDFS_3029);
108.        ctrl_3029.add(btnReset_3029);
109.

```

```

110.         north_3029.add(title_3029, BorderLayout.NORTH);
111.         north_3029.add(ctrl_3029, BorderLayout.SOUTH);
112.         add(north_3029, BorderLayout.NORTH);
113.     }
114.
115.     private JButton buatTombol_3029(String teks_3029, Color
        warna_3029) {
116.         JButton btn_3029 = new JButton(teks_3029);
117.         btn_3029.setBackground(warna_3029);
118.         btn_3029.setForeground(Color.WHITE);
119.         btn_3029.setFont(new Font("Dialog", Font.BOLD,
            13));
120.         btn_3029.setFocusPainted(false);
121.         return btn_3029;
122.     }
123.
124.     private void buildGraph_3029() {
125.         taGraph_3029 = new JTextArea();
126.         taGraph_3029.setFont(new Font("Monospaced",
            Font.PLAIN, 13));
127.         taGraph_3029.setEditable(false);
128.         taGraph_3029.setBackground(Color.WHITE);
129.         taGraph_3029.setText(displayGraph_2511533029());
130.
131.         JScrollPane sp_3029 = new
            JScrollPane(taGraph_3029);
132.         sp_3029.setBorder(BorderFactory.createTitledBorder(
            BorderFactory.createLineBorder(Color.GRAY),
133.         "VISUALISASI GRAPH", TitledBorder.LEFT,
134.         TitledBorder.TOP,
            new Font("Dialog", Font.BOLD, 12)));
135.         sp_3029.setPreferredSize(new Dimension(750, 240));
136.         add(sp_3029, BorderLayout.CENTER);
137.     }
138.
139.
140.     private void buildResult_3029() {
141.         taResult_3029 = new JTextArea(6, 70);
142.         taResult_3029.setFont(new Font("Monospaced",
            Font.BOLD, 13));
143.         taResult_3029.setEditable(false);
144.         taResult_3029.setBackground(Color.WHITE);
145.         taResult_3029.setText(
146.             "Hasil Pencarian :\n" +
147.             "Jalur :\n" +
148.             "Node Dikunjungi :\n" +
149.             "Jumlah Node Dikunjungi : 0"
150.         );
151.         JScrollPane sp_3029 = new
            JScrollPane(taResult_3029);
152.         sp_3029.setBorder(BorderFactory.createLineBorder(Color.GRAY));
153.         add(sp_3029, BorderLayout.SOUTH);
154.     }
155.

```

```

156.         // ===== displayGraph =====
157.         public String displayGraph_2511533029() {
158.             return
159.                 "\n" +
160.                 "    Gerbang ----- PosSatpam -----
161.     Perpustakaan ----- LabKomputer\n" +
162.                 "        |                               |
163.     | \n" +
164.                 "        |                               |
165.     | \n" +
166.                 "    Lapangan           KantorGuru
167.     Kelas10           Kelas11\n" +
168.                 "        |                               |
169.     | \n" +
170.                 "        |                               |
171.     | \n" +
172.                 "    Kantin -----
173.     -----> Kelas12\n" +
174.                 " \n" +
175.                 "    Total Node: 10 | Total Edge: 15\n" +
176.                 "    Peta: Jalur Antar Lokasi di Sekolah\n";
177.         }
178.
179.         // ===== BFS =====
180.         public List<Integer> BFS_2511533029(int start_3029, int
181.         goal_3029) {
182.             boolean[] visited_3029 = new
183.             boolean[nodes_3029.length];
184.             int[] parent_3029 = new
185.             int[nodes_3029.length];
186.             List<Integer> order_3029 = new ArrayList<>();
187.             Arrays.fill(parent_3029, -1);
188.
189.             Queue<Integer> queue_3029 = new LinkedList<>();
190.             queue_3029.add(start_3029);
191.             visited_3029[start_3029] = true;
192.
193.             while (!queue_3029.isEmpty()) {
194.                 int curr_3029 = queue_3029.poll();
195.                 order_3029.add(curr_3029);
196.                 if (curr_3029 == goal_3029) break;
197.                 for (int[] e_3029 : edges_3029) {
198.                     int nb_3029 = tetangga_3029(e_3029,
199.                     curr_3029, visited_3029);
200.                     if (nb_3029 != -1) {
201.                         visited_3029[nb_3029] = true;
202.                         parent_3029[nb_3029] = curr_3029;
203.                         queue_3029.add(nb_3029);
204.                     }
205.                 }
206.             }
207.
208.             displayPath_2511533029(start_3029, goal_3029,
209.             parent_3029, order_3029, "BFS");
210.             return bangunJalur_3029(goal_3029, parent_3029);

```

```

198.     }
199.
200.     // ===== DFS =====
201.     public List<Integer> DFS_2511533029(int start_3029, int
goal_3029) {
202.         boolean[] visited_3029 = new
boolean[nodes_3029.length];
203.         int[] parent_3029 = new
int[nodes_3029.length];
204.         List<Integer> order_3029 = new ArrayList<>();
205.         Arrays.fill(parent_3029, -1);
206.
207.         Stack<Integer> stack_3029 = new Stack<>();
208.         stack_3029.push(start_3029);
209.
210.         while (!stack_3029.isEmpty()) {
211.             int curr_3029 = stack_3029.pop();
212.             if (visited_3029[curr_3029]) continue;
213.             visited_3029[curr_3029] = true;
214.             order_3029.add(curr_3029);
215.             if (curr_3029 == goal_3029) break;
216.             for (int[] e_3029 : edges_3029) {
217.                 int nb_3029 = tetangga_3029(e_3029,
curr_3029, visited_3029);
218.                 if (nb_3029 != -1) {
219.                     parent_3029[nb_3029] = curr_3029;
220.                     stack_3029.push(nb_3029);
221.                 }
222.             }
223.         }
224.         displayPath_2511533029(start_3029, goal_3029,
parent_3029, order_3029, "DFS");
225.         return bangunJalur_3029(goal_3029, parent_3029);
226.     }
227.
228.     private int tetangga_3029(int[] edge_3029, int
curr_3029, boolean[] visited_3029) {
229.         if (edge_3029[0] == curr_3029
&& !visited_3029[edge_3029[1]]) return edge_3029[1];
230.         if (edge_3029[1] == curr_3029
&& !visited_3029[edge_3029[0]]) return edge_3029[0];
231.         return -1;
232.     }
233.
234.     private List<Integer> bangunJalur_3029(int goal_3029,
int[] parent_3029) {
235.         List<Integer> path_3029 = new ArrayList<>();
236.         for (int at_3029 = goal_3029; at_3029 != -1;
at_3029 = parent_3029[at_3029])
237.             path_3029.add(0, at_3029);
238.         return path_3029;
239.     }
240.
241.     // ===== displayPath =====

```

```

242.         public void displayPath_2511533029(int start_3029, int
goal_3029, int[] parent_3029,
243.                                     List<Integer>
order_3029, String algo_3029) {
244.             List<Integer> path_3029 =
bangunJalur_3029(goal_3029, parent_3029);
245.             boolean found_3029 = !path_3029.isEmpty() &&
path_3029.get(0) == start_3029;
246.
247.             StringBuilder sb_3029 = new StringBuilder();
248.             sb_3029.append("Hasil Pencarian :
").append(algo_3029).append("\n");
249.
250.             sb_3029.append("Jalur           : ");
251.             if (found_3029) {
252.                 for (int i_3029 = 0; i_3029 < path_3029.size();
i_3029++) {
253.                     sb_3029.append(nodes_3029[path_3029.get(i_3029)]);
254.                     if (i_3029 < path_3029.size() - 1)
sb_3029.append(" -> ");
255.                 }
256.             } else {
257.                 sb_3029.append("Tidak ditemukan");
258.             }
259.
260.             sb_3029.append("\nNode Dikunjungi : ");
261.             for (int i_3029 = 0; i_3029 < order_3029.size();
i_3029++) {
262.                 sb_3029.append(nodes_3029[order_3029.get(i_3029)]);
263.                 if (i_3029 < order_3029.size() - 1)
sb_3029.append(", ");
264.             }
265.
266.             sb_3029.append("\nJumlah Node Dikunjungi :
").append(order_3029.size());
267.             taResult_3029.setText(sb_3029.toString());
268.         }
269.
270.         // ===== resetGraph =====
271.         public void resetGraph_2511533029() {
272.             taGraph_3029.setText(displayGraph_2511533029());
273.             taResult_3029.setText(
274.                 "Hasil Pencarian :\n" +
275.                 "Jalur :\n" +
276.                 "Node Dikunjungi :\n" +
277.                 "Jumlah Node Dikunjungi : 0"
278.             );
279.         }
280.
281.         public static void main(String[] args_3029) {
282.             SwingUtilities.invokeLater(PetaSekolah_2511533029::new);

```

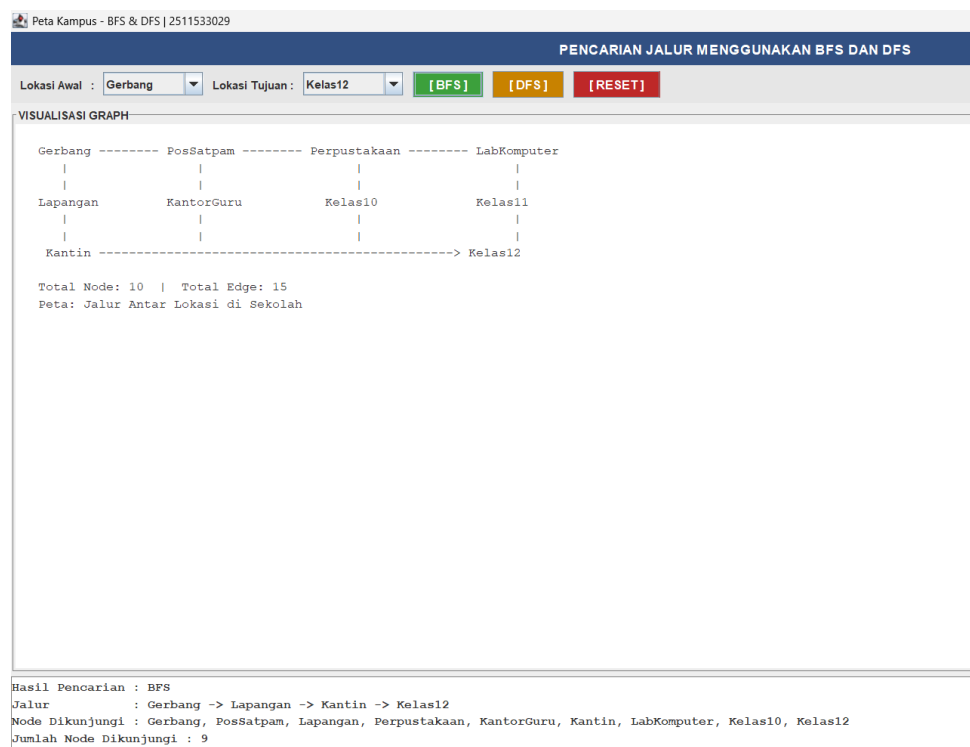
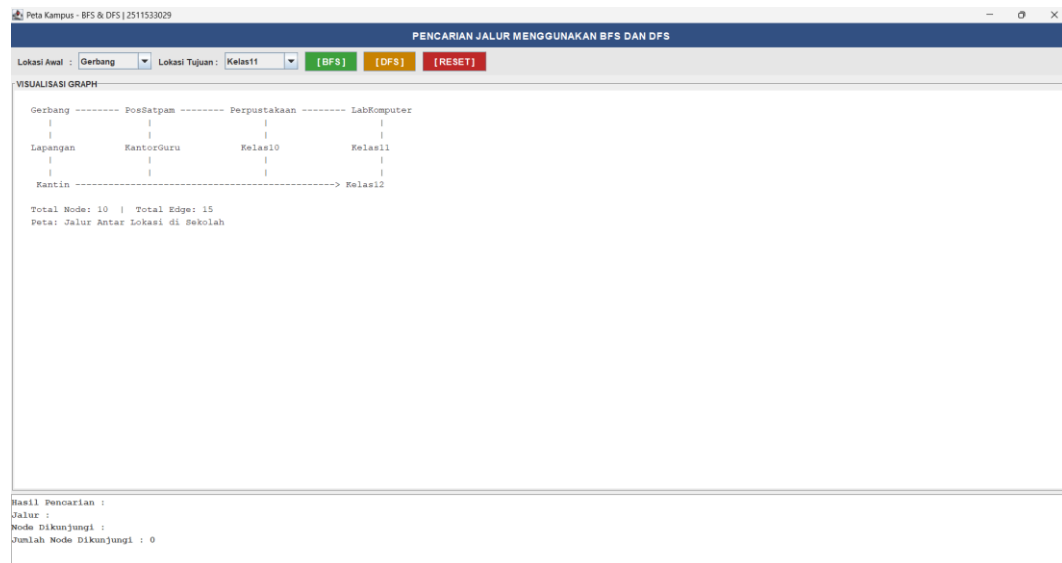
```

283.         }
284.     }

```

2. Output

Berikut beberapa output dari kode program



Peta Kampus - BFS & DFS | 2511533029

PENCARIAN JALUR MENGGUNAKAN BFS DAN DFS

Lokasi Awal : Lokasi Tujuan :

VISUALISASI GRAPH

```

Gerbang ----- PosSatpam ----- Perpustakaan ----- LabKomputer
|               |               |               |
|               |               |               |
Lapangan       KantorGuru       Kelas10       Kelas11
|               |               |               |
|               |               |               |
Kantin -----> Kelas12

Total Node: 10 | Total Edge: 15
Peta: Jalur Antar Lokasi di Sekolah

```

Hasil Pencarian : DFS
 Jalur : PosSatpam -> KantorGuru -> Perpustakaan -> Kelas10 -> Kantin -> Kelas12
 Node Dikunjungi : PosSatpam, KantorGuru, Perpustakaan, Kelas10, Kantin, Kelas12
 Jumlah Node Dikunjungi : 6